

Name : Adhip Halder

University Roll : 18871024030

Stream : MCA

Year : 2nd

Semester : 3rd

Subject Name : Software Engineering using UML

Subject Code : MCAN301

❏ **About Software Engineering**

➤ **Definition**

Software Engineering is a disciplined approach to the development, operation, and maintenance of software using engineering principles.

➤ **Objectives**

- Build reliable, scalable, and efficient software systems
- Improve productivity and reduce development time
- Ensure high-quality deliverables that meet customer needs

➤ **Principles of Software Engineering**

- Modularity : Breaking down into manageable parts
- Abstraction : Simplifying complex processes
- Reusability : Promoting reuse of components
- Scalability : Handling increased load efficiently
- Maintainability : Easy to update or fix

➤ **Common Software Engineering Models:**

- ✓ Waterfall Model
- ✓ Agile Model
- ✓ Spiral Model
- ✓ V-Model

❏ Phases in Software Engineering

➤ Requirement Gathering & Analysis

- Identify user needs, functional and non-functional requirements
- Techniques: Interviews, Questionnaires, Use Cases

➤ System Design

- Architecture design, data flow diagrams, database schema
- High-level and low-level design

➤ Implementation (Coding)

- Programming the design into working software
- Follows coding standards and language-specific conventions

➤ Testing

- Unit Testing, Integration Testing, System Testing, Acceptance Testing
- Ensures the system meets requirements

➤ Deployment

- Software is installed and made operational for users

➤ Maintenance

- Bug fixes, updates, and feature enhancements

❏ What is a Use Case Diagram?

➤ Definition

A Use Case Diagram models the interactions between users and the system to achieve specific goals.

➤ Why Use It?

- ✓ Provides a high-level functional view
- ✓ Easy to understand for both technical and non-technical stakeholders
- ✓ Helps identify system boundaries and required features

➤ Elements of a Use Case Diagram:

- **Actors:** External entities interacting with the system (e.g., User, Admin)
- **Use Cases:** Specific actions/functions the system performs
- **System Boundary:** Defines what is included in the system
- **Relationships:** Include, Extend, Generalization

➤ Benefits

- Improves requirement clarity
- Aids in system documentation
- Useful in testing and validation

❑ How to Draw a Use Case Diagram

Example System: Library Management System

➤ Identify Actors

- Example: Librarian, Member (Student/Teacher), System Administrator

➤ Define Use Cases

- Issue Book
- Return Book
- Borrow Book
- Add New Book
- Manage Memberships

➤ Draw System Boundary

- A box labeled “*Library System*” containing all use cases

➤ Connect Actors to Use Cases

- Librarian → Issue Book, Return Book, Add New Book
- Member → Borrow Book, Return Book
- Admin → Manage Memberships

